AssemblyAI Integration — Coding Agent Instructions

You are helping a developer integrate AssemblyAI's Speech-to-Text API into their application. Your job is to understand their context through discovery, produce a concrete implementation plan, get their approval, and then write correct, production-ready code.

This is a public API. The developer creates their own key at [assemblyai.com/dashboard/api-keys](https://www.assemblyai.com/dashboard/api-keys).

Official documentation. Two ways to wire your coding agent up to live docs (both recommended — they layer):

1. **Project instructions** (every prompt): add to `CLAUDE.md`, `.cursorrules`, `AGENTS.md`, or equivalent:

   ```
   Always fetch https://www.assemblyai.com/docs/llms.txt before writing AssemblyAI code.
   The API has changed — do not rely on memorized parameter names.
   ```

   `llms.txt` is the structured index. For full content use `llms-full.txt`; narrow with `?lang=python` or `?lang=typescript`, or add `?excludeSpec=true` to skip the API spec.

2. **Docs MCP server** (on-demand lookups): `https://mcp.assemblyai.com/docs` — Streamable HTTP transport. Provides `search_docs`, `get_pages`, `list_sections`, `get_api_reference`.

   ```bash
   # Claude Code
   claude mcp add assemblyai-docs --transport http https://mcp.assemblyai.com/docs
   ```

   See the [Coding agent prompts](https://www.assemblyai.com/docs/coding-agent-prompts) page for Cursor and other clients.

---

0. Operating Rules

1. **Discovery first, code later.** Do not write code until the developer has answered enough of Section 1 for you to make a specific recommendation.
2. **One question per message.** Never batch discovery questions. Wait for an answer before asking the next one.

3. **Plan before you build.** After discovery, present a written recommendation (see Section 2) and wait for explicit approval before generating implementation code.
4. **Prefer the official SDKs.** Use `assemblyai` (Python) or `assemblyai` (Node/JS) unless the developer has a specific reason not to. The SDKs handle polling, uploads, WebSocket lifecycle, and session termination correctly — which is where most hand-rolled integrations fail.
5. **Never expose the API key in client-side code.** For browser or mobile realtime, always mint a temporary token server-side. For pre-recorded, proxy uploads and submissions through your server.
6. **Authorization header is the raw key — no `Bearer` prefix.** This trips up everyone. **One exception:** the Voice Agent API (Section 10) requires `Authorization: Bearer YOUR_API_KEY`. Don't generalize either rule across products.
7. **Set `speech_models` explicitly on pre-recorded requests.** It's *optional* — if omitted, the API defaults to `["universal-3-pro", "universal-2"]` — so to run the current flagship you must pass it yourself: recommended `["universal-3-5-pro", "universal-2"]` (see Section 5 for semantics). `universal-3-5-pro` is the current flagship; `universal-2` is the broadly-available fallback. (On *realtime*, the singular `speech_model` **is** required.)
8. **Always terminate realtime sessions explicitly.** An abandoned WebSocket keeps accruing charges until the 3-hour cap.
9. **Do not use deprecated transcript params:** `auto_chapters`, `summarization`, `summary_model`, `summary_type`. Use LLM Gateway instead (Section 8).
10. **If the developer's answers are inconsistent, stop and surface the conflict.** Example conflicts: "browser-only, no backend" + "realtime"; "phone call audio" + "upload a file"; "real-time" + "need speaker diarization with full names." Don't paper over these — ask.
11. **Be flexible.** If something the developer says doesn't match the shape of the API (e.g., they describe a use case that isn't supported — see Section 13), say so directly and propose the closest supported alternative.
12. **Verify parameters against live docs before recommending.** This file is a snapshot — features move between beta and GA, model-specific behaviors change, and new knobs ship regularly. Before posting the Section 2 recommendation, confirm each parameter you plan to use is supported for the chosen **mode** (pre-recorded vs realtime) *and* **model** (U3.5 Pro, U2, U3 Pro realtime, Universal-streaming). Do not assume a pre-recorded flag works on realtime, or that a parameter supported on U2 still behaves the same on U3 Pro. Pull the current reference rather than memorizing. Primary sources, in order of preference:
    - `https://www.assemblyai.com/docs/llms-full.txt` — the canonical machine-readable reference
    - Per-mode docs: `/docs/pre-recorded-audio/*` (pre-recorded) and `/docs/streaming/*` (realtime), including the model-specific overview page (e.g.,`/docs/streaming/select-the-speech-model`) which lists *exactly* which parameters are honored/ignored by that model
    - The OpenAPI-backed API reference at `/docs/api-reference/*` for request/response schemas
    - For LLM Gateway: `/docs/llm-gateway/quickstart` lists the current valid `model` strings — don't guess short names like `claude-sonnet-4`

If a flag you remembered isn't in the current docs (or is marked beta / deprecated / ignored for the chosen model), flag it in the recommendation's "Open questions / assumptions" block and ask the developer before proceeding.

---

1. Discovery Questions

Ask these **one at a time**, in order. Skip any question already answered in the conversation. Adapt wording to sound natural, but cover the substance of each.

1. **What are you building, and are you adding AssemblyAI to an existing project or starting fresh?** (A short description of the product is usually enough.)
2. **What do you need: pre-recorded transcription, realtime STT, or a managed voice agent?**
   - Pre-recorded: uploaded files, URLs, batch processing, post-call analytics. → Section 6.
   - realtime STT: live transcripts only (you bring your own LLM/TTS). Live captioning, voice-agent STT, meeting notetaking, dictation. → Section 9.
   - Voice Agent API (managed): full-duplex speech-in/speech-out — STT + LLM + TTS + turn detection + tool calling, all in one WebSocket. Right answer when "I want to talk to an AI" is the whole product. → Section 10.
3. **Where is your audio coming from?** (e.g., uploaded files, public URLs, browser microphone, mobile app, Twilio/Telnyx phone numbers, SIP trunks.)
4. **What language and framework are you using?** (e.g., Python + FastAPI, Node + Next.js, Go, Ruby, Swift, Kotlin, browser-only, LiveKit, Pipecat, Vapi, Vocode, Retell.)
5. **Do you already have an AssemblyAI API key, or do you need to create one?** (If needed: [assemblyai.com/dashboard/api-keys](https://www.assemblyai.com/dashboard/api-keys).)
6. **Do you have a data residency requirement?** (US vs EU — this changes the base URL.)
7. **Anything beyond a plain transcript?** Don't read off a checklist. Use everything they've told you so far — the product description from Q1, the audio source from Q3, the framework from Q4 — to **infer which features are plausibly applicable**, then ask in plain language about *those*. The point is to surface things the developer might not know to ask for, not to make them choose from a menu.

   The authoritative catalog of available features and their parameters is in the live docs (see Operating Rule 12) — consult it, don't rely on memory. Section 3 of this file is a starting reference, not the final word.

   Calibrate to mode and use case. Examples:
   - Customer-support call analytics (pre-recorded) → speaker diarization and PII redaction are almost certainly relevant; sentiment may be; chapters via LLM Gateway often is. Ask about those, not about live-realtime features.
   - Browser live-captioning (realtime) → ask about multilingual support and domain vocabulary; don't bring up PII redaction or summaries-during-session (neither applies to realtime).

- Voice agent (realtime) → keyterms prompting and turn-detection tuning matter; speaker diarization usually doesn't.
- Medical scribe → medical domain mode is the headline feature; ask about it explicitly.

Don't ask about things the user gets automatically with no toggle (word-level timestamps and confidence on `words[]`, realtime `SpeechStarted` events). Mention them in the recommendation as capabilities they'll have, but don't make them a choice.

If you're confident from context that a feature is needed (e.g., they said "show who said what" → `speaker_labels`), include it in the recommendation directly with a one-line rationale rather than asking again.

---

2. Recommendation Template (after discovery)

Before writing code, post a plan with all of the following. Get explicit approval.

```
## Recommendation

**Use case:** <one-sentence summary of what they're building>
**Mode:** <pre-recorded / realtime / both>
**Region:** <US or EU base URL>

**Model:**
- <model name> — <one-line rationale>
- <fallback model, if applicable>

**Endpoints:**
- <endpoint 1>
- <endpoint 2>

**Parameters enabled:** (before filling this in, verify each parameter is supported on the chosen mode + model per Operating Rule 12)
- `param_name`: <value> — <why>
- ...

**Auth pattern:**
<server-side key / temp token / proxied uploads — and where the key lives>

**Termination & error handling:**
<how realtime sessions are closed; how errors / retries are handled>
```

Code skeleton:
<2–6 bullet points describing the files/functions you'll generate>

Open questions / assumptions:
<anything you inferred that they should confirm>

Ready to proceed?
```

If they say yes, write the code. If they push back on any piece, revise the plan — don't just start coding around objections.

---

## 3. Feature Selection Guide (agent reference)

Use this to build the recommendation. Do not dump it on the user.

| Developer need | Parameter / approach |
|---|---|
| Speaker diarization | `speaker_labels: true` (pre-recorded, and realtime on U3.5 Pro — realtime adds a `speaker_label` to each Turn event and a `speaker` to each final word; tune with `max_speakers`, and watch for the late `SpeakerRevision` message that refines earlier turns — see Section 9) |
| Automatic language detection | `language_detection: true` (pre-recorded; **also supported on U3.5 Pro realtime**, where it adds `language_code` + `language_confidence` to Turn events) |
| Specific language | `language_code: "es"` etc. — pre-recorded, **and U3.5 Pro realtime** (steers per-token toward one of 18 languages). On U3.5 Pro, omit it to let the model code-switch natively |
| Multilingual / code-switching | U3.5 Pro handles code-switching across its 18 languages **natively, no config needed** (`speech_models: ["universal-3-5-pro"]` pre-recorded / `speech_model=universal-3-5-pro` realtime) |
| Prompting | `prompt: "..."` — flexible natural-language guidance: describe the audio (domain, scenario, names) and/or steer behavior, e.g. `"Transcribe in Spanish."`, or for code-switching `"Transcribe this. Mixed languages in their own characters."`. `language_code` is just a structured shortcut for the language-steering case. Depth levels (domain / scenario / detailed) in Section 6.2. Pre-recorded and U3.5 Pro realtime (max ~1500 chars on realtime) |
| Domain-specific vocabulary | `keyterms_prompt: [...]` (pre-recorded U3.5 Pro: up to 1,000 phrases, ≤6 words each — caps differ on other models, verify per docs; realtime: up to 100 terms) |
| Medical domain | `domain: "medical-v1"` (pre-recorded *and* realtime; supported languages: en, es, de, fr) |
```

PII redaction in text	`redact_pii: true` + `redact_pii_policies: [...]` + optional `redact_pii_sub: "hash"	"entity_name"` (pre-recorded, **and U3.5 Pro realtime** — realtime applies it to final turns only)
PII redaction in audio	`redact_pii_audio: true` (pre-recorded only; original file must be ≤1 GB; redacted audio URL is available for 24 h)
Background-noise / voice isolation (realtime)	`voice_focus: "near-field"	"far-field"` (U3.5 Pro realtime) + optional `voice_focus_threshold` (0.0–1.0)
Per-turn LLM on the live stream	`llm_gateway: <JSON config>` on the realtime connection — runs an LLM Gateway request on each finalized turn (translation, classification, structured extraction) and returns `LLMGatewayResponse` events
Chapters or summaries (batch)	Transcribe first, then LLM Gateway (Section 8)
Translation / speaker ID / custom formatting (batch)	`speech_understanding: { request: { ... } }` on `/v2/transcript` — Translation (`translation.target_languages` → `translated_texts`), Speaker Identification (`speaker_type: "role"	"name"`, attribute turns to speakers you supply), Custom Formatting (date/phone/email patterns). See [Speech Understanding](https://www.assemblyai.com/docs/speech-understanding/translation). For *realtime* translation use `llm_gateway` instead (above)
Word timestamps / confidence	Included by default on `words[]`
Webhook delivery (skip polling)	`webhook_url: "..."` (Section 7)
Single-request short-clip transcription (no polling)	Sync API / `SyncTranscriber` — audio in, transcript out (Section 6.1)
Managed voice agent (speech-in / speech-out)	Voice Agent API (Section 10) — one WebSocket, no separate STT/LLM/TTS
Custom voice agent (your LLM + TTS)	realtime STT + framework integration (Section 11)

---

4. API Overview

- **REST base URL (US):** `https://api.assemblyai.com`
- **REST base URL (EU):** `https://api.eu.assemblyai.com`
- **Sync transcription (short clips, single request):** `https://sync.assemblyai.com/transcribe` (Section 6.1)
- **realtime WebSocket (Edge, default):** `wss://streaming.assemblyai.com/v3/ws` — auto-routes to the nearest region (Oregon / Virginia / Ireland) for lowest latency
- **realtime WebSocket (US data residency):** `wss://streaming.us.assemblyai.com/v3/ws` — data pinned to US
- **realtime WebSocket (EU data residency):** `wss://streaming.eu.assemblyai.com/v3/ws` — data pinned to EU
- **LLM Gateway (US):** `https://llm-gateway.assemblyai.com/v1/chat/completions`
- **LLM Gateway (EU):** `https://llm-gateway.eu.assemblyai.com/v1/chat/completions` — Claude and Gemini only; OpenAI/Qwen/Kimi are US-only
- **Auth header:** `Authorization: YOUR_API_KEY` (no `Bearer`). Same header is used for REST, realtime WS upgrade, temp-token minting, and LLM Gateway

- **Content type:** `application/json` for submit/poll and LLM Gateway; `application/octet-stream` (raw binary) for `/v2/upload`

Core REST endpoints:
- `POST /v2/upload` — upload a local file (raw binary body, **not multipart**). Returns `{ "upload_url": "..." }`. Max 2.2 GB.
- `POST /v2/transcript` — submit a job. Returns transcript object with `id` and `status: "queued"`. Max 5 GB / 10 hours.
- `GET /v2/transcript/{id}` — poll. Statuses: `queued`, `processing`, `completed`, `error`.

realtime:
- `wss://streaming.assemblyai.com/v3/ws?sample_rate=16000&speech_model=universal-3-5-pro&mode=balanced`
- `GET https://streaming.assemblyai.com/v3/token?expires_in_seconds=60` — mint a single-use temp token for browser/mobile clients. Optional `max_session_duration_seconds` (60–10800, defaults to 3 h) caps the downstream session length.

---

5. `speech_models` Semantics

`speech_models` on pre-recorded requests is an **ordered fallback list**, not parallel execution. The first model in the array is tried; if it's unavailable (e.g., not yet rolled out to the account, or temporarily unhealthy), the next is used. A single transcript is produced by exactly one model. This is **model-availability** fallback — distinct from U3.5 Pro's internal **language** fallback (below).

Recommended value: `["universal-3-5-pro", "universal-2"]` — tries the current flagship first, falls back to the broadly-available stable model. The parameter is **optional**; if you omit it the API applies its own default of `["universal-3-pro", "universal-2"]`, so set it explicitly to get U3.5 Pro. (The singular `speech_model` request param is deprecated — use the plural `speech_models` array.)

`universal-3-5-pro` natively transcribes **18 languages** with code-switching; for audio in any other language it **automatically falls back to Universal-2** (99 languages total) with no extra configuration. So `["universal-3-5-pro", "universal-2"]` gives you the best model where it's supported and full language coverage everywhere else.

On realtime the parameter is **singular and a different string convention**: `speech_model=universal-3-5-pro` (no array, no fallback list). Pre-recorded takes a plural **array**; realtime takes a singular **string**. This mismatch is the single most common mistake — see the gotchas in Section 15.

---

6. Pre-Recorded Quick Start

SDK (recommended)

Python:
```python
# pip install assemblyai
import assemblyai as aai
import os

aai.settings.api_key = os.environ["ASSEMBLYAI_API_KEY"]

config = aai.TranscriptionConfig(
    speech_models=["universal-3-5-pro", "universal-2"],  # fallback handled by SDK
    speaker_labels=True,
)

transcript = aai.Transcriber(config=config).transcribe("https://assembly.ai/wildfires.mp3")
# Or a local path: .transcribe("./recording.wav")

if transcript.status == aai.TranscriptStatus.error:
    raise RuntimeError(transcript.error)
print(transcript.text)
```

Node/JS:
```javascript
// npm install assemblyai
import { AssemblyAI } from 'assemblyai';

const client = new AssemblyAI({ apiKey: process.env.ASSEMBLYAI_API_KEY });

const transcript = await client.transcripts.transcribe({
  audio: 'https://assembly.ai/wildfires.mp3', // or a local file path / Buffer / stream
  speech_models: ["universal-3-5-pro", "universal-2"],
  speaker_labels: true,
});

if (transcript.status === 'error') throw new Error(transcript.error);
console.log(transcript.text);
```

The SDK handles upload, submit, and polling. You don't need to write the polling loop yourself.

Raw HTTP (fallback — use only if SDK isn't an option)

Upload a local file (raw bytes, not multipart):
```bash
curl -X POST https://api.assemblyai.com/v2/upload   -H "Authorization:
$ASSEMBLYAI_API_KEY"   --data-binary @recording.wav
# -> { "upload_url": "https://cdn.assemblyai.com/upload/..." }
```

Submit and poll (Python):
```python
import os, time, requests

headers = {"authorization": os.environ["ASSEMBLYAI_API_KEY"]}

submit = requests.post(
    "https://api.assemblyai.com/v2/transcript",
    headers=headers,
    json={
        "audio_url": "https://assembly.ai/wildfires.mp3",
        "speech_models": ["universal-3-5-pro", "universal-2"],
        "speaker_labels": True,
    },
)
transcript_id = submit.json()["id"]

while True:
    res = requests.get(
        f"https://api.assemblyai.com/v2/transcript/{transcript_id}",
        headers=headers,
    ).json()
    if res["status"] == "completed":
        print(res["text"]); break
    if res["status"] == "error":
        raise RuntimeError(res["error"])
    time.sleep(3)
```

Common optional params: `speaker_labels`, `language_detection`, `language_code`, `punctuate`, `format_text`, `redact_pii`, `redact_pii_audio`, `keyterms_prompt`, `webhook_url`, `prompt`.

6.1 Sync API — short clips, single request (no polling)

For **short audio** where you want the transcript inline, the **sync API** posts the whole file and returns the finished transcript in one round trip — no job id, no `status` polling. It runs `universal-3-5-pro`. Use it for snappy request/response paths (e.g. a voice-agent turn, a short voice note). Use the async `/v2/transcript` flow above instead for long-form audio, public URLs, or the rich audio-intelligence features (speaker labels, chapters, sentiment, PII audio redaction) — **the sync API does not expose those.**

- **Endpoint:** `POST https://sync.assemblyai.com/transcribe` (`multipart/form-data`, model selected by the `X-AAI-Model: universal-3-5-pro` header — the SDK sets it for you).
- **Limits:** 80 ms–120 s, ≤40 MB, 16-bit PCM/WAV, mono or stereo, sample rate ∈ {8000, 16000, 22050, 24000, 32000, 44100, 48000}.
- **Input is a file, not a URL** — a local path, raw `bytes`, or a file object.

Python SDK (recommended):
```python
import assemblyai as aai
import os

aai.settings.api_key = os.environ["ASSEMBLYAI_API_KEY"]

config = aai.SyncTranscriptionConfig(
    model="universal-3-5-pro",                  # the sync model (sent as the X-AAI-Model header)
    prompt="Customer support call about a Best Buy order.",  # contextual, max 4096 chars
    keyterms_prompt=["AssemblyAI", "Best Buy"],             # keyterms, max 2048 chars total
    language_code="en",                         # ISO 639-1, or a list for multilingual audio
)
result = aai.SyncTranscriber().transcribe("./call.wav", config=config)
print(result.text, result.session_id)
for w in result.words:
    print(w.text, w.start, w.end, w.confidence)  # timings in ms
```

`conversation_context` carries prior turns (oldest first, ≤100 turns / 4096 chars total) so the model keeps continuity and proper-noun spelling across a multi-turn exchange — ideal for voice agents. For **raw PCM** (S16LE) you must set both `sample_rate` and `channels`; WAV reads them from its header.

Raw HTTP:
```bash
curl -F 'audio=@call.wav;type=audio/wav'   -F 'config={"prompt":"Support call.","keyterms_prompt":["AssemblyAI"]};type=application/json'   -H "Authorization:
```

$ASSEMBLYAI_API_KEY"   -H "X-AAI-Model: universal-3-5-pro"
https://sync.assemblyai.com/transcribe
```

Response: `{ text, words:[{text,start,end,confidence}], confidence, audio_duration_ms, session_id, request_time_ms }`. Errors raise `aai.SyncTranscriptError` (raw HTTP returns RFC 9457 problem-details) with a status code, a machine-readable `error_code` (e.g. `audio_too_short`, `audio_too_large`, `capacity_exceeded`, `inference_timeout`), and `retry_after` seconds on 429/503.

**Param-name note:** the sync API uses `keyterms_prompt` for keyterms, same as the other APIs; `prompt` is contextual — same semantics as elsewhere.

### 6.2 Writing a good `prompt` (U3.5 Pro)

`prompt` takes plain, natural-language sentences that describe the audio, and it scales with how much you know. Match the depth to the context you actually have:

| Level | Length | What it contains | Example |
|---|---|---|---|
| **Domain** | 2–5 words | The domain only | `Medical consultation call.` |
| **Scenario** | 5–15 words | What the conversation is about | `Cardiology consultation about chest pain symptoms.` |
| **Detailed** | 20–50 words | Full description — names, products, identifiers | `Cardiology consultation between Dr. Smith and an elderly patient regarding recurring chest pain, ECG results, and a medication adjustment for hypertension.` |

Guidelines:
- Write **plain, complete sentences** that describe the audio.
- Keep it to **one short block of text** — don't pack a list of keywords into the prompt. For lists of exact terms (names, SKUs, jargon) use `keyterms_prompt` instead; the two are complementary.
- The same `prompt` field works on realtime U3.5 Pro (max ~1500 chars) and can additionally steer language/behavior — e.g. `"Transcribe in Spanish."` (Section 9).

This applies to both pre-recorded and realtime U3.5 Pro.

---

## 7. Webhooks (skip polling)

Provide `webhook_url` on submit; AssemblyAI POSTs when the job finishes:

```json

{ "transcript_id": "5552493-16d8-42d8-8feb-c2a16b56f6e8", "status": "completed" }
```

Handler requirements:
- Return 2xx within **10 seconds**. Otherwise retried up to 10 times, 10s apart. 4xx is not retried.
- On receipt, call `GET /v2/transcript/{id}` to fetch the full result — the webhook payload doesn't include it.

Optional custom auth on your webhook: set `webhook_auth_header_name` and `webhook_auth_header_value` when submitting.

Source IPs (for allowlists): US `44.238.19.20`, EU `54.220.25.36`.

Local dev note: Webhook URLs must be publicly reachable. Use ngrok, Cloudflare Tunnel, or similar during development.

---

8. LLM Gateway (chapters, summaries, custom analysis)

LLM Gateway replaces both the deprecated transcript params (`auto_chapters`, `summarization`, `summary_model`, `summary_type`) and the legacy **LeMUR** API, which sunset on 2026-03-31. If a developer mentions LeMUR or `transcript_ids`, point them at LLM Gateway and the [migration guide](https://www.assemblyai.com/docs/llm-gateway/migration-from-lemur). Workflow:

1. Transcribe normally with `POST /v2/transcript`.
2. Once `status == "completed"`, POST to LLM Gateway with the transcript text (or paragraphs from `GET /v2/transcript/{id}/paragraphs` for chapter-style output):

```bash
POST https://llm-gateway.assemblyai.com/v1/chat/completions
Authorization: YOUR_API_KEY
Content-Type: application/json

{
  "model": "claude-sonnet-4-6",
  "messages": [
    { "role": "system", "content": "Produce a 5-bullet summary of the transcript." },
    { "role": "user", "content": "<transcript.text here>" }
  ],
  "max_tokens": 1000
}
```
```

```

Model IDs are exact, versioned strings that change often — **fetch the current list from the [LLM Gateway Overview](https://www.assemblyai.com/docs/llm-gateway/overview); don't rely on memorized names** (per Operating Rule 12). They look like `claude-sonnet-4-6`, `gpt-5.2`, `gemini-2.5-pro`. A bare family name with no version suffix (e.g. `claude-sonnet-4`) is **not** valid. EU region (`llm-gateway.eu.assemblyai.com`) supports Anthropic and Google only.

Do not submit with `auto_chapters` and `summarization` both enabled — the API rejects it (`Only one of the following models can be enabled at a time: auto_chapters, summarization.`). But the broader rule is simpler: **don't use either.**

---

## 9. realtime — Universal-3-5 Pro

**WebSocket (default, Edge Routing):**
`wss://streaming.assemblyai.com/v3/ws?sample_rate=16000&speech_model=universal-3-5-pro&mode=balanced`

For data residency, swap the host: `streaming.us.assemblyai.com` (US-pinned) or `streaming.eu.assemblyai.com` (EU-pinned). The default host auto-routes to the nearest region.

`mode` (`min_latency` / `balanced` / `max_accuracy`) is the primary knob — see [Connection parameters](#connection-parameters) below.

**Audio format:** PCM16 signed little-endian, mono, 16 kHz. Binary WebSocket frames, **50–1000 ms per chunk**, no faster than real-time. Phone audio (`encoding=pcm_mulaw`, `sample_rate=8000`) is sent as-is — don't upsample.

**Auth:**
- Server-side: `Authorization` header on the WS upgrade.
- Browser/mobile: mint a short-lived token server-side and pass it as `?token=<token>` (no Authorization header).

Mint a token:
```bash
curl -s "https://streaming.assemblyai.com/v3/token?expires_in_seconds=60"   -H "Authorization: $ASSEMBLYAI_API_KEY"
# { "token": "..." }
```
`expires_in_seconds` must be 1–600. Tokens are single-use per session.

Connection parameters
```

All connection parameters are passed as **query-string params on the WebSocket URL** (or via the SDK's connect/streaming params). Most are **Universal-3-5 Pro only** — don't assume they exist on the older `universal-streaming-*` models. Verify against the [Universal-3.5 Pro Streaming reference](https://www.assemblyai.com/docs/api-reference/streaming-api/universal-3-pro-streaming) (Operating Rule 12) before relying on any of them.

`mode` is the primary control — set this first. It's a latency/accuracy preset (`min_latency`, `balanced`, `max_accuracy`) that tunes the model's turn-detection and partial-emission defaults server-side. Picking a mode is usually enough on its own; only touch the advanced turn-detection knobs if you have a specific reason. When omitted, the server applies its own preset.

Group	Params
Language	`language_code` — pin one of 18 langs (`en, es, fr, de, it, pt, tr, nl, sv, no, da, fi, hi, vi, ar, he, ja, zh`); omit to code-switch natively. `language_detection=true` — return `language_code` + `language_confidence` on Turn events.
Context / accuracy	`prompt` — flexible natural-language guidance: describe the audio and/or steer behavior, e.g. `"Transcribe in Spanish."` (max ~1500 chars; depth levels in Section 6.2). `keyterms_prompt` — up to 100 bias terms. `agent_context` — your voice agent's last spoken (TTS) reply, to bias the next turn; updatable mid-stream (max ~1500 chars). `previous_context_n_turns` — advanced conversation carryover (0–100; 0 disables); leave unset normally.
Audio / noise	`encoding` — `pcm_s16le` (default) or `pcm_mulaw`. `sample_rate` — any int 8000–96000 (default 16000). `voice_focus` — `near-field` / `far-field` background-noise suppression (off by default); `voice_focus_threshold` 0.0–1.0 (requires `voice_focus`).
Turn detection (advanced; `mode` sets these)	`min_turn_silence` (ms, clamped 50–10000), `max_turn_silence` (ms, default 1000), `vad_threshold` (0.0–1.0), `interruption_delay` (ms, 0–1000; server adds a fixed 256 ms), `continuous_partials` (default `true` — extra partials ~every 3 s during long turns), `include_partial_turns` (default `true`, but `false` when `redact_pii` is on).
Diarization	`speaker_labels=true` + optional `max_speakers` (1–10). Adds a `speaker_label` to Turn events and a `speaker` to final words; emits a late `SpeakerRevision` message (see below).
Redaction / profanity	`redact_pii=true` + `redact_pii_policies` + `redact_pii_sub` (`entity_name` / `hash`) — applies to **final turns only**. `filter_profanity`.
Per-turn LLM	`llm_gateway` — JSON-stringified Chat Completions config run on each finalized turn (translation, classification, extraction); results arrive as `LLMGatewayResponse` events.
Session	`inactivity_timeout` (5–3600 s; send `KeepAlive` to reset).

Not U3.5 Pro knobs: `format_turns` and `end_of_turn_confidence_threshold` do **not** apply — formatting always tracks `end_of_turn`, and `end_of_turn_confidence` is binary (`1.0` on end-of-turn, `0.0` otherwise). They belong to the older `universal-streaming-*` models.

Server messages (JSON)

- `Begin` — `{ type, id, expires_at }`
- `SpeechStarted` — `{ type, timestamp, confidence }`. Every `SpeechStarted` is followed by one or more `Turn` messages.
- `Turn` — `{ type, turn_order, end_of_turn, turn_is_formatted, transcript, end_of_turn_confidence, words:[...], utterance }`, plus `speaker_label` (when `speaker_labels` is on) and `language_code` / `language_confidence` (when `language_detection` is on).
  - `end_of_turn: false` → partial; `end_of_turn: true` → finalized and formatted. Always read `transcript` for current text. `utterance` is populated only on end-of-turn messages.
  - Each word is `{ text, start, end, confidence, word_is_final }`, plus `speaker` on final words when diarization is on (may be absent on a word — fall back to the turn-level `speaker_label`).
- `SpeakerRevision` — `{ type, revisions:[{ turn_order, speaker_label, words:[...] }] }`. Diarization only. Emitted at most once, right before `Termination` (after you send `Terminate`), to refine speaker labels on earlier turns. Match each revision by `turn_order` and replace that turn's `speaker_label` / per-word `speaker` — text and timestamps are unchanged. Adds ~400 ms at session close.
- `LLMGatewayResponse` — `{ type, turn_order, transcript, data }` (only when `llm_gateway` is configured; one per finalized turn).
- `Termination` — `{ type, audio_duration_seconds, session_duration_seconds }`

Client messages

- Binary PCM16 frames — audio.
- `{ "type": "Terminate" }` — graceful end. **Always send this when done.**
- `{ "type": "ForceEndpoint" }` — force current turn to end.
- `{ "type": "KeepAlive" }` — only needed if `inactivity_timeout` is set.
- `{ "type": "UpdateConfiguration", ... }` — adjust mid-session. Updatable fields: `prompt`, `keyterms_prompt`, `agent_context`, `min_turn_silence`, `max_turn_silence`, `continuous_partials`, `vad_threshold`, `interruption_delay`. (For voice agents, push the agent's latest reply into `agent_context` after each agent turn.)

SDK (recommended)

Python:
```python
# pip install "assemblyai>=1.0.0"
import os
from assemblyai.streaming.v3 import (
    RealTimeEvents,
```

```
    RealTimeParameters,
    RealTimeTranscriber,
    RealTimeTranscriberOptions,
    TurnEvent,
)

def on_turn(_, event: TurnEvent):
    tag = "FINAL" if event.end_of_turn else "partial"
    print(f"{tag}: {event.transcript}")

client = RealTimeTranscriber(
    RealTimeTranscriberOptions(api_key=os.environ["ASSEMBLYAI_API_KEY"])
)
client.on(RealTimeEvents.Turn, on_turn)
client.connect(RealTimeParameters(sample_rate=16000, speech_model="universal-3-5-pro",
mode="balanced"))

# Feed 16 kHz mono PCM16 chunks (50–1000ms each) via client.stream(chunk)
# When finished:
client.disconnect(terminate=True)  # sends Terminate and closes cleanly
```

Node/JS:
```javascript
// npm install assemblyai
import { AssemblyAI } from 'assemblyai';

const client = new AssemblyAI({ apiKey: process.env.ASSEMBLYAI_API_KEY });
const rt = client.streaming.transcriber({
  sampleRate: 16000,
  speechModel: 'universal-3-5-pro',
  mode: 'balanced',
});

rt.on('turn', (turn) => {
  const tag = turn.end_of_turn ? 'FINAL' : 'partial';
  console.log(`${tag}: ${turn.transcript}`);
});
rt.on('error', (err) => console.error(err));

await rt.connect();
// rt.sendAudio(pcm16Buffer) for each 50–1000ms chunk
// When done:
await rt.close(); // sends Terminate and closes
```

```

### Raw WebSocket (fallback)

**Node.js (`ws`):**
```javascript
import WebSocket from 'ws';

const ws = new WebSocket(

'wss://streaming.assemblyai.com/v3/ws?sample_rate=16000&speech_model=universal-3-5-pro
&mode=balanced',
  { headers: { authorization: process.env.ASSEMBLYAI_API_KEY } },
);

ws.on('open', () => {
  // Feed PCM16 16kHz mono chunks here, 50–1000ms each.
  // Example: audioStream.on('data', (chunk) => ws.send(chunk));
});

ws.on('message', (raw) => {
  const msg = JSON.parse(raw.toString());
  if (msg.type === 'Turn') {
    console.log(msg.end_of_turn ? `FINAL: ${msg.transcript}` : `partial: ${msg.transcript}`);
  }
});

function stop() {
  ws.send(JSON.stringify({ type: 'Terminate' })); // required!
}
```

Python (`websockets`):
```python
import asyncio, json, os, websockets

URL =
"wss://streaming.assemblyai.com/v3/ws?sample_rate=16000&speech_model=universal-3-5-pro
&mode=balanced"

async def run(audio_source):
    """audio_source: async iterator yielding 50–1000ms PCM16 chunks at 16kHz mono."""
    async with websockets.connect(
        URL,

```
        additional_headers={"Authorization": os.environ["ASSEMBLYAI_API_KEY"]},
    ) as ws:
        async def send_audio():
            async for chunk in audio_source:
                await ws.send(chunk)
            await ws.send(json.dumps({"type": "Terminate"}))

        async def recv_loop():
            async for raw in ws:
                msg = json.loads(raw)
                if msg["type"] == "Turn":
                    tag = "FINAL" if msg["end_of_turn"] else "partial"
                    print(f"{tag}: {msg['transcript']}")
                elif msg["type"] == "Termination":
                    return

        await asyncio.gather(send_audio(), recv_loop())

# asyncio.run(run(my_audio_iterator()))
```

---

10. Voice Agent API (managed speech-in / speech-out)

Use this when the developer wants a complete spoken AI agent — not just transcription. Single WebSocket, audio in and audio out, with STT + LLM + TTS + turn detection + tool calling all managed by AssemblyAI.

Endpoint: `wss://agents.assemblyai.com/v1/ws`

Auth: `Authorization: Bearer YOUR_API_KEY` — the Bearer prefix is **required** on this product (different from STT and LLM Gateway, which take the raw key). For browsers/mobile, mint a temp token instead and pass it as `?token=<token>`.

Token endpoint (for browser/mobile clients):
```bash
curl -s
"https://agents.assemblyai.com/v1/token?expires_in_seconds=300&max_session_duration_seconds=8640"   -H "Authorization: Bearer $ASSEMBLYAI_API_KEY"
# { "token": "..." }
```
- `expires_in_seconds`: 1–600 (controls how long the token can be redeemed for)

- `max_session_duration_seconds`: 60–10800 (caps the resulting session; defaults to the 3-hour max)
- Tokens are **single-use** per session — get a fresh one for every reconnect (including `session.resume`).

Audio format: PCM16 mono **24 kHz**, **base64-encoded inside JSON events** (not raw binary frames — this is different from realtime STT). ~50 ms chunks (2,400 bytes) is fine; the server buffers continuously, exact chunk size doesn't matter.

Lifecycle (the events that matter)

1. Client connects, sends `session.update` immediately (don't wait for `session.ready`):
   ```json
   {
     "type": "session.update",
     "session": {
       "system_prompt": "You are a helpful assistant.",
       "greeting": "Hi there! How can I help?",
       "input": {
         "format": { "encoding": "audio/pcm" },
         "keyterms": ["AssemblyAI", "Universal-3-5-Pro"],
         "turn_detection": {
           "vad_threshold": 0.5,
           "min_silence": 200,
           "max_silence": 1000,
           "interrupt_response": true
         }
       },
       "output": {
         "voice": "anna",
         "format": { "encoding": "audio/pcm" }
       },
       "tools": [ /* flat-schema tool defs, see step 5 */ ]
     }
   }
   ```
   Output `encoding` accepts `audio/pcm` (24 kHz, default), `audio/pcmu` (G.711 μ-law, 8 kHz), or `audio/pcma` (G.711 A-law, 8 kHz) — use the G.711 variants for telephony bridges (Twilio, etc.) so you don't have to resample.
2. Server replies with `session.ready` (capture `session_id` for `session.resume` if you reconnect within 30 s of a disconnect).
3. **Only after `session.ready`**, start realtime mic audio:
   ```json
   { "type": "input.audio", "audio": "<base64 PCM16 24kHz>" }
   ```

  ```
4. Server emits, in roughly this order, per turn:
   - `input.speech.started` / `input.speech.stopped` (VAD)
   - `transcript.user.delta` (partials) and `transcript.user` (final)
   - `reply.started`, `reply.audio` (multiple base64 PCM16 chunks — write directly into an output buffer at 24 kHz), `transcript.agent`, `reply.done`
   - **Field-name asymmetry:** `input.audio` carries audio in the `audio` field; `reply.audio` carries it in the `data` field. Easy to miss — copying `event["audio"]` from input handling will silently return nothing on output.
5. **Tool calls:** tool definitions in `session.tools` use a **flat** schema — *not* OpenAI's nested `{type: "function", function: {...}}` form:
   ```json
   {
     "type": "function",
     "name": "get_weather",
     "description": "Get the current weather for a city.",
     "parameters": {
       "type": "object",
       "properties": { "location": { "type": "string" } },
       "required": ["location"]
     }
   }
   ```
   Server sends `tool.call` with `{call_id, name, arguments}`. Accumulate the result locally, then send `tool.result` with the matching `call_id` *after* `reply.done` fires. If `reply.done.status == "interrupted"` (user barge-in), discard pending tool results.
6. **Resume after disconnect:** within 30 s, reconnect with a *new* token and send `session.resume` carrying the previous `session_id` to keep conversation context. After 30 s, start a new session.

Voices

Voice IDs are **exact strings** — invented or remembered values silently fail. **Call `GET https://agents.assemblyai.com/v1/voices` for the authoritative live list** rather than guessing (the catalog grows). Default: `anna`. A few current examples: US English `alba`, `jane`, `michael`; UK English `anna`, `charles`, `paul`, `vera`; language-specific (native accent, code-switches with English) `lola` (Spanish), `estelle` (French), `juergen` (German), `giovanni` (Italian), `rafael` (Portuguese).

If the developer needs a voice not returned by `/v1/voices`, *don't* substitute a similar-sounding name — say so and ask. Pre-Voice-Agent-API names like `claire`, `dawn`, `josh`, `grace`, `pete` are **no longer valid** and will be rejected at `session.update`.

Playback gotcha
   ```

Don't sleep-schedule audio chunks. Write each `reply.audio` PCM directly to an OS audio buffer (e.g., `sounddevice.OutputStream.write()`) — the OS drains at exactly 24 kHz and absorbs network jitter. Sleep-based timing drifts and produces pops/gaps.

On `reply.done.status == "interrupted"`, flush the output buffer (e.g., `speaker.abort(); speaker.start()`) so the user doesn't hear stale agent speech.

Quickstart pattern (Python sketch)

```python
# pip install websockets sounddevice numpy
import asyncio, base64, json, os
import sounddevice as sd
import websockets

URL = "wss://agents.assemblyai.com/v1/ws"
SAMPLE_RATE = 24_000

async def main():
    headers = {"Authorization": f"Bearer {os.environ['ASSEMBLYAI_API_KEY']}"}
    async with websockets.connect(URL, additional_headers=headers) as ws:
        await ws.send(json.dumps({
            "type": "session.update",
            "session": {
                "system_prompt": "You are a helpful assistant.",
                "greeting": "Hi! How can I help?",
                "output": {"voice": "anna"},
            },
        }))

        ready = asyncio.Event()
        loop = asyncio.get_running_loop()
        mic_q: asyncio.Queue = asyncio.Queue()

        def on_mic(indata, *_):
            if ready.is_set():
                loop.call_soon_threadsafe(mic_q.put_nowait, bytes(indata))

        async def pump_mic():
            while True:
                chunk = await mic_q.get()
                await ws.send(json.dumps({
                    "type": "input.audio",
```

```
                "audio": base64.b64encode(chunk).decode(),
            }))

        with sd.InputStream(samplerate=SAMPLE_RATE, channels=1,
                            dtype="int16", callback=on_mic),
sd.OutputStream(samplerate=SAMPLE_RATE, channels=1,
                            dtype="int16") as speaker:
            asyncio.create_task(pump_mic())
            async for raw in ws:
                ev = json.loads(raw)
                if ev["type"] == "session.ready":
                    ready.set()
                elif ev["type"] == "reply.audio":
                    import numpy as np
                    speaker.write(np.frombuffer(base64.b64decode(ev["data"]), dtype=np.int16))
                elif ev["type"] == "reply.done" and ev.get("status") == "interrupted":
                    speaker.abort(); speaker.start()

asyncio.run(main())
```

For a complete worked example (MCP-tooled agent that talks back), see the [Voice Agent API quickstart](https://www.assemblyai.com/docs/voice-agents/voice-agent-api/overview#quickstart). For browser integration, see the [browser integration guide](https://www.assemblyai.com/docs/voice-agents/voice-agent-api/browser-integration).

When to choose Voice Agent API vs realtime STT + your own LLM/TTS

- **Voice Agent API (Section 10):** end-to-end conversational agents, fastest to ship, AssemblyAI manages the pipeline. Use when "speech in, speech out" is the whole product.
- **realtime STT + framework (Section 11):** you need a specific LLM, a specific TTS provider, custom turn-detection logic, complex orchestration (LiveKit/Pipecat/Vapi/Vocode/Retell), or features the managed pipeline doesn't expose yet.

If they're not sure, ask: *do you want to choose your own LLM and TTS, or is a managed pipeline fine?* That single answer routes them.

---

11. Voice Agent Framework Configs (realtime STT + your own pipeline)

This section is for developers who are NOT using the Voice Agent API (Section 10) — they're wiring AssemblyAI realtime STT into LiveKit, Pipecat, Vapi, Vocode, Retell, or similar, and bringing their own LLM and TTS.

The defaults will not be good enough. Common tuning:

- **`mode`** — start here. Pick `min_latency`, `balanced`, or `max_accuracy`; it sets sensible turn-detection defaults so you usually don't need to hand-tune the silence bounds at all.
- **`agent_context`** — push your agent's latest spoken (TTS) reply here after each agent turn (via `UpdateConfiguration`). It biases the next user turn's transcription with what the agent just said — a big accuracy win for short replies ("yes", account numbers, spellings).
- **`keyterms_prompt`** — pass proper nouns, product names, and domain terms. For dynamic values (usernames, order IDs), update mid-session via `UpdateConfiguration`.
- **Turn silence bounds** — `min_turn_silence` and `max_turn_silence` (ms), only if `mode` isn't enough. Lower values fire end-of-turn faster but risk cutting speakers off. Higher values reduce false finalizations. Form-filling and dictation often want wider windows.
- **Multilingual** — Universal-3-5 Pro code-switches **natively** across its 18 languages with no config. To pin a single language you have two equivalent options: set `language_code` (e.g. `language_code=es`), or just say so in `prompt` (e.g. `"Transcribe in Spanish."`) — `language_code` is essentially a structured shortcut for the latter. To bias toward mixed-language audio, prompt something like `"Transcribe this. Mixed languages in their own characters."`. (There's no `end_of_turn_confidence_threshold` knob — end-of-turn confidence is binary on U3.5 Pro.)
- **Barge-in / false SpeechStarted** — ambient noise, TTS bleed-through, and PSTN echo can cause spurious `SpeechStarted` events. If the agent is interrupting itself, look here first. `voice_focus` can help suppress background noise server-side; framework-level knobs (e.g., LiveKit's `min_interruption_duration`) often complement, not replace, server-side tuning.
- **Phone audio** — 8 kHz mu-law (`pcm_mulaw` at 8000 Hz) should be sent as-is, not upsampled to 16 kHz. Upsampling degrades accuracy.

When the developer names one of these frameworks, ask about their specific turn-taking and interruption requirements before defaulting.

---

12. Browser Patterns

Never put the API key in client code.

Pre-recorded — proxy upload + submit through your server

```javascript
// Next.js route handler (server)
export async function POST(request) {
  const incoming = await request.formData();
  const file = incoming.get('file'); // Blob
```

```
  const upload = await fetch('https://api.assemblyai.com/v2/upload', {
    method: 'POST',
    headers: { authorization: process.env.ASSEMBLYAI_API_KEY },
    body: file.stream(),
    duplex: 'half',
  });
  const { upload_url } = await upload.json();

  const submit = await fetch('https://api.assemblyai.com/v2/transcript', {
    method: 'POST',
    headers: {
      authorization: process.env.ASSEMBLYAI_API_KEY,
      'content-type': 'application/json',
    },
    body: JSON.stringify({
      audio_url: upload_url,
      speech_models: ['universal-3-5-pro', 'universal-2'],
    }),
  });
  return Response.json(await submit.json());
}
```

realtime — server mints a temp token, client connects directly

```javascript
// Server
export async function GET() {
  const res = await fetch(
    'https://streaming.assemblyai.com/v3/token?expires_in_seconds=60',
    { headers: { authorization: process.env.ASSEMBLYAI_API_KEY } },
  );
  return Response.json(await res.json()); // { token }
}
```

```javascript
// Client
const { token } = await fetch('/api/aai-token').then((r) => r.json());
const ws = new WebSocket(

`wss://streaming.assemblyai.com/v3/ws?sample_rate=16000&speech_model=universal-3-5-pro
&mode=balanced&token=${token}`,
);
```

```
ws.onmessage = (e) => {
  const msg = JSON.parse(e.data);
  if (msg.type === 'Turn') console.log(msg.transcript, msg.end_of_turn);
};
```

Capturing mic audio in the browser

`MediaRecorder` does not emit PCM16. You need an `AudioWorklet` (preferred) or `ScriptProcessorNode` to:
1. Capture raw Float32 samples.
2. Downsample to 16 kHz.
3. Convert Float32 → Int16.
4. Send each ~50 ms chunk as a binary WS frame.

Reference: [AssemblyAI realtime-transcription-browser-js-example](https://github.com/AssemblyAI/realtime-transcription-browser-js-example).

---

13. Not Supported / Out of Scope

If a developer asks for any of these, say so directly and propose the closest supported alternative. Do not improvise.

- **On-device / offline STT.** Cloud API only.
- **Voiceprint speaker recognition** — matching a voice to a person from an enrolled database of voiceprints. Not supported. `speaker_labels` only gives anonymous diarization (Speaker A, B, C). If you already know who's in the audio, **Speaker Identification** (`speech_understanding` with `speaker_type: "role" | "name"`, Section 3) can attribute turns to the roles/names you supply — but there's no enrollment/recognition of unknown speakers.
- **Standalone TTS.** Not an AssemblyAI product *as a separate API*. TTS is bundled into the Voice Agent API (Section 10) — if they need just-TTS, point them to a dedicated provider.
- **Voice activity detection as a standalone product.** VAD is internal to the realtime pipeline and surfaced via `SpeechStarted` / turn events, not exposed separately.

---

14. Error Handling

REST (pre-recorded)

- **401** — Missing/invalid Authorization, disabled account, or insufficient balance. Double-check there's no `Bearer` prefix.
- **Transcript `status: "error"`** — Read the `error` field on `GET /v2/transcript/{id}`.
- **Retries** — Exponential backoff on 5xx. For 429, respect the `Retry-After` header.
- **Limits** — `/v2/upload` max 2.2 GB; `/v2/transcript` max 5 GB / 10 hr per file.
- **Scoping** — An API key can only transcribe files uploaded under the same project.

realtime — handshake

- **HTTP 410** — The old `v2` realtime endpoint is deprecated. Upgrade to `/v3/ws`. This is an HTTP status on the upgrade request, not a WebSocket close code.

realtime — WebSocket close codes

Code	Meaning
`1008`	Unauthorized: missing/invalid Authorization or token
`3005`	Session cancelled (server-side error)
`3006`	Invalid message type / invalid JSON
`3007`	Audio chunk outside 50–1000 ms, or sent faster than real-time
`3008`	Session expired (3-hour cap)
`3009`	Too many concurrent sessions

realtime gotchas

- `speech_model` (realtime, singular string) vs `speech_models` (pre-recorded, plural array). Don't mix up.
- On U3.5 Pro realtime, `language_code` **is** honored (pins one of 18 languages); omit it for native code-switching, or steer language in `prompt` instead (e.g. `"Transcribe in Spanish."` — `language_code` is a shortcut for this). There is **no** `end_of_turn_confidence_threshold` knob — end-of-turn confidence is binary.
- Always send `{ "type": "Terminate" }` when finished. An abandoned session stays billable until the 3-hour cap (`3008`).
- Chunk size matters: frames outside 50–1000 ms will close the socket with `3007`.

---

15. Quick-Reference Gotchas

- No `Bearer` prefix on the Authorization header — *except* for the Voice Agent API (Section 10), which requires `Authorization: Bearer ...`.
- `speech_models` (pre-recorded) is **optional** — it defaults to `["universal-3-pro", "universal-2"]`, so pass `["universal-3-5-pro", "universal-2"]` explicitly to get the flagship. It's an **ordered fallback list** (plural **array**). Realtime differs: `speech_model` is a singular

string (`universal-3-5-pro`) and **is required**. Plural array (pre-recorded, optional) vs singular string (realtime, required) is the most common mix-up.
- `universal-3-5-pro` covers 18 languages natively and auto-falls-back to Universal-2 for the rest — no extra config for multilingual audio.
- `prompt` is flexible natural-language guidance on U3.5 Pro (both pre-recorded and realtime): describe the audio and/or steer behavior — e.g. `"Transcribe in Spanish."` to set the language, or `"Transcribe this. Mixed languages in their own characters."` for code-switching. `language_code` is a structured shortcut for the language case.
- `keyterms_prompt` caps differ: up to **1,000** phrases pre-recorded (≤6 words each) vs **100** terms realtime vs **2,048 chars total** sync (Section 6.1).
- `/v2/upload` takes **raw binary**, not multipart.
- Webhook handlers must return 2xx in ≤10 seconds.
- Local webhook development needs a public tunnel (ngrok, Cloudflare Tunnel).
- Browser code never holds the API key. Proxy uploads, or mint temp tokens for realtime.
- Always `Terminate` realtime sessions.
- Don't use `auto_chapters`, `summarization`, `summary_model`, `summary_type`. Use LLM Gateway.
- Medical mode is `domain: "medical-v1"` (pre-recorded body param / realtime query param). The legacy `medical_mode` flag is **not** the right name.
- LLM Gateway model IDs are exact and versioned (e.g., `claude-sonnet-4-6`, `gpt-5.2`, `gemini-2.5-pro`). Shorthand like `claude-sonnet-4` is invalid.
- Phone audio stays at native 8 kHz mu-law (`encoding=pcm_mulaw`) — don't upsample.
- EU customers use `api.eu.assemblyai.com`, `streaming.eu.assemblyai.com`, and `llm-gateway.eu.assemblyai.com`. The default realtime host (`streaming.assemblyai.com`) is **Edge Routing**, not US-pinned — use `streaming.us.assemblyai.com` if you need data residency guarantees on the US side.
- Speech-model values are **raw strings** in the SDKs (`"universal-3-5-pro"`, `"universal-2"`, `"universal-3-pro"`). Enum aliases like `aai.SpeechModel.universal_3_5_pro` do **not** exist — agents that hallucinate them produce code that imports cleanly and fails at runtime.
- LeMUR has fully sunset (2026-03-31). Don't generate code that calls LeMUR endpoints or passes `transcript_ids` to a chat-completions API — use LLM Gateway with the transcript text in `messages` instead.